Geofencing, ML and Zone-Authority Audit — Tourist Safety Monitoring

Date: 18 October 2026 **Scope:** `D:\TouristSafetyMonitoring-src` (Android Compose app + Python/SQLite authority backend + `ml/` training pipeline) **Method:** read the shipped source and shipped model assets, changed the code in place, ran the real test suites, ran a live HTTP probe against the running backend, and rebuilt the APK. Nothing below is estimated unless it is explicitly labelled as such.

Everything in this document is either **measured this round**, **read from a shipped file**, or **documented in the repo**. Where something is not done, it says so.

---

0. Executive summary

#	Question asked	Finding	Action taken
1	Is a low ML score hiding geographical danger?	**No data path existed, but the presentation was ambiguous.** The ML score never fed the geofence verdict; however the top-of-screen banner was driven by `snapshot.overallStatus`, which reads SAFE (green) when tracking is stopped or has no fix — so a tourist standing inside a restricted zone with tracking stopped saw a green "Safe".	Added **Card 1 (Geographical safety)** fed only by position + zones, and re-pointed the banner at it. A restricted verdict can no longer coexist with a green banner.
2	Are zones owned by the authority?	**No.** Zones lived only in the phone's local Room DB, and **the tourist could create, edit, disable and delete them from the UI**. The backend had **no zone API at all** (confirmed by exhaustive route enumeration).	Removed every tourist zone control; zone screens are now officer-gated; added an **authority-only zone API** with server-side role enforcement, validation and change history.
3	Does the app and the backend use the same zone data?	**Not yet.** The backend is now the authoritative store, and the app can read it, but the app's tracking still evaluates the local Room copy. Wiring the pull/push needs a verified local-schema migration.	Backend done and tested; the app-side read client is done and tested; the Room sync is **explicitly not done** (see §7).
4	Does the future-risk model have a valid future-fall target?	**Yes**, and it is unusually well-constructed: the positive class is a *real recorded impact* 1–50 samples (1.0 s) ahead of the window end.	Documented exactly (§5).
5	Why does the displayed probability vary (≈5.4 / 5.6 / 9.2 %)?	Different live 1-second windows, and the number is a **calibrated** transform of a forest vote share.	Exact formula and arithmetic consequence given (§5.4).

---

1. Part A — Three separate concepts

The app now keeps three outputs apart, as three cards on the tourist screens.

Card	Question it answers	Source of truth	Type
1. Geographical safety (new)	"Am I inside an area the authority has closed off?"	GPS position + authority zones, via `GeoFenceEngine`	Deterministic geometry — **not** a model
2. Movement / activity	"What is the person doing right now?"	24-feature forest on the accelerometer/gyro window (UCI HAR)	Trained model
3. Future risk prediction	"Is a hard fall impact likely within 1.0 s?"	22-feature forest on the accelerometer (UniMiB SHAR)	Trained model

Card 1 is built by `GeoSafetyCardModel.build(snapshot, maxUsableAccuracyMeters, nowMillis)`. It takes **no model state as an argument**. That is what makes "a Low prediction cannot suppress the geographical warning" structural rather than a promise, and it is pinned by a regression test that builds a Low future-risk card for the same instant and asserts the geographical card is still an unqualified red inside-zone verdict (`GeoSafetyCardModelTest.a low fall-risk prediction does not suppress or soften the inside-zone warning`).

Card 1 states (exhaustive)

State	When	Tone
`INSIDE_RESTRICTED_ZONE`	A current, usable fix is inside an ACTIVE restricted or dangerous polygon	Red, with the required sentence
`NEAR_BOUNDARY_UNCERTAIN`	The fix is inside the tolerance band, its uncertainty circle straddles an edge, or accuracy is worse than the usable limit	Amber
`OUTSIDE_ZONES`	A current usable fix, outside every restricted/dangerous polygon	Green
`CHECKING_LOCATION`	Tracking is running but there is no fix yet	Amber ("we have not decided")
`LOCATION_UNAVAILABLE`	Tracking stopped, or the last fix is older than 60 s	Amber

The mandatory wording shown whenever a decided fix is inside:

> **YOU ARE INSIDE A RESTRICTED AREA. Move to a safer location if you can do so safely.**

Together with the zone name, the zone type, **distance to that zone's boundary** (not to its centre), the GPS accuracy and the position time.

Card 1 also carries a sentence that keeps it apart from the model cards: "*This is decided from your GPS position and the zones the authority has set. It is not a fall-risk score, and the fall-risk model cannot change it.*"

---

2. Part B — The GPS → geofence → UI trace, and the defects found

2.1 The path

```
FusedLocationSource  →  TrackingController.onReading()
   → GeoFenceEngine.evaluate(reading, zones)         (per-zone ZoneStatus)
   → TrackingSnapshot.overallStatus / zoneStatuses   (SafetyStatus.worst reduction)
   → StatusBanner + GeoSafetyCard                    (both read the same snapshot)
```

`GeoFenceEngine` uses ray-casting containment in the lon/lat plane (`PolygonGeometry.contains`), with boundary and uncertainty handling in a local metre frame (`LocalFrame` , equirectangular about the polygon mean). Membership is **hysteretic** — entry tolerance 8 m, exit tolerance 15 m — and BOUNDARY / UNCERTAIN are **sticky**: they never flip membership. `maxUsableAccuracyMeters = 100 m` .

Verified by inspection and by the existing suite (`GeoFenceEngineTest` , `PolygonContainmentServiceTest` , `PolygonGeometryTest` , `GeofenceWalkthroughTest` , `PolygonValidatorTest`), all passing:

- containment is order-independent and winding-independent;
- a point exactly on an edge is not spuriously "inside";
- distance is reported to the **boundary**, not the centroid;
- an accuracy worse than the usable limit yields UNCERTAIN rather than a guess;
- only `enabled` zones are evaluated (`GeoFenceDao.observeEnabled`).

2.2 Defect confirmed and fixed: the banner could show green while the verdict was unknown

Root cause. `HomeScreen` and `TrackingScreen` rendered `statusVisualsFor(snapshot.overallStatus)` . When tracking is stopped, or no fix has arrived, `overallStatus` keeps its default/previous value, which reduces to **SAFE** — a green "Safe" banner reading "*You are outside every monitored zone.*" A tourist physically inside a restricted zone, with tracking paused, therefore saw green. This is exactly the "geographical danger is not made sufficiently clear" symptom.

Fix. Added `geoSafetyVisualsFor(state)` and re-pointed both banners at `GeoSafetyCardModel.build(...).state` . Green `Outside restricted zones` is now reachable **only** with a current, usable fix that is demonstrably outside every restricted/dangerous polygon. With no fix the banner reads "*Checking your location*" or "*Location unavailable*" in amber.

2.3 Defect confirmed: zones were phone-local and tourist-mutable

Root cause. The backend has **no zone storage and no zone routes** — an exhaustive enumeration of `@bp.*` / `@app.*` / `add_url_rule` in `backend/tsm/api.py` plus `create_app` in `backend/tsm/__init__.py` returns 19 routes, none of them zone CRUD. Zones lived only in the app's Room table `geo_fences` , and `ManageZonesScreen` / `GeoFenceEditorScreen` were reachable from the tourist bottom bar ("Create", "Zones").

Consequence. A tourist could define, disable or delete the very restricted area they were supposed to be warned by; and an authority-created zone could not exist at all. There was no server-side authorisation to enforce because there was no server-side zone object.

Fix. See Part C.

2.4 GPS-uncertainty handling — no change needed

The containment pipeline already fails safe: coarse accuracy → UNCERTAIN (sticky), stale/no fix → no verdict, boundary band → NEAR_BOUNDARY (sticky). The gap was purely **presentational** (§2.2), and that is what was fixed.

2.5 Lat/lon order and the GeoJSON convention

The app stores vertices as named `latitude` / `longitude` pairs (`PolygonCodec` → `lat,lon;…`), never as positional arrays, and the new backend stores JSON objects with the same named fields. There is therefore no place where a GeoJSON `[lon, lat]` array could be silently transposed, and the new backend parser rejects a non-numeric or out-of-range coordinate by name (`vertex 2 longitude is outside [-180, 180]`). A regression test asserts the order survives a round trip (`AuthorityZoneApiTest.latitude and longitude are not swapped`).

---

3. Part C — Zone management is now authority-only, enforced server-side

3.1 Tourist UI

- Bottom bar is now **Home · Tracking · Alerts · Settings**. The "Create" and "Zones" tabs are gone (`AppNavigation.bottomDestinations` , derived from `TouristNavigation.tabRoutes`).
- `HomeScreen` no longer takes `onCreateZone` / `onOpenZones` and no longer renders "Create zone" / "Manage". Its zone card is read-only and states that restricted areas are created and managed by the authority.
- The editor and manage screens still exist, but are gated on an authenticated officer session and are reached only from the officer console. Without a session they render an "Authority only" screen.
- `TouristNavigationTest` fails if a zone-management route is re-added to the tourist tabs.

This is a **usability** gate. The real boundary is server-side.

3.2 Backend — new authority-owned zone resource

New tables (additive, `CREATE TABLE IF NOT EXISTS` , so existing databases upgrade without touching data):

- `zones` — `id, name, description, zone_type, vertices (JSON ring), vertex_count, status, revision, created_by, created_at, updated_by, updated_at, published_at, archived_at`
- `zone_history` — append-only `zone_id, revision, action, creator, at, snapshot`

New routes in `backend/tsm/api.py` :

Method	Path	Guard	Purpose
`GET`	`/api/v1/zones`	`@require_auth()`	The ACTIVE zones a phone evaluates. Readable by any signed-in subject (a phone needs the geometry to warn its user).
`GET`	`/api/v1/authority/zones`	`@require_auth("authority")`	Every zone incl. drafts/archived; optional `?status=` .
`GET`	`/api/v1/authority/zones/<id>`	`@require_auth("authority")`	One zone plus its last 50 history entries.
`POST`	`/api/v1/authority/zones`	`@require_auth("authority")`	Create.
`PUT`	`/api/v1/authority/zones/<id>`	`@require_auth("authority")`	Edit (bumps `revision`).

Method	Path	Guard	Purpose
`POST`	`/api/v1/authority/zones/<id>/status`	`@require_auth("authority")`	Activate / deactivate / archive.
`DELETE`	`/api/v1/authority/zones/<id>`	`@require_auth("authority")`	**Soft** delete (archive). The row and its history survive.

Statuses: `ACTIVE` (published — phones evaluate it), `INACTIVE` (draft/paused — no phone evaluates it), `ARCHIVED` (kept for history, never evaluated).

Validation rejects: fewer than 3 or more than 500 vertices, non-object vertices, non-numeric or out-of-range coordinates, unknown `zoneType`, blank/oversized name, and **degenerate (zero-area) rings** — area is the shoelace area in a local metre frame anchored on the polygon mean.

3.3 Live end-to-end evidence (real HTTP, running backend, 18 Oct 2026)

```
officer login   -> 200
tourist login   -> 200

officer creates zone          -> 201   zoneId=1 status=ACTIVE revision=1 vertexCount=4
tourist reads active zones    -> 200   (['PROBE quarry'])
tourist tries to create       -> 403   {"error":"forbidden","message":"This endpoint is for role 'authority
tourist tries to delete       -> 403
no token creates              -> 401
officer creates degenerate    -> 400   {"error":"bad_request","message":"the polygon is degenerate: it encl
officer pauses zone           -> 200   status=INACTIVE
tourist read after pause      -> []
officer archives zone         -> 200   status=ARCHIVED
officer reads history         -> 200   actions=['ARCHIVE', 'STATUS', 'CREATE']
tourist read after archive    -> []
```

The tourist token is refused on every mutating route; the read route is a different path from every write route.

> **Disclosure.** This probe wrote one zone to the **dev** database (`backend/var/authority.db`) and then archived it, leaving an `ARCHIVED` "PROBE quarry" row plus its three history rows. Nothing was deleted and no other row was touched. No production database exists or was involved.

3.4 DB "migrations"

The backend has no migration tooling by design (it must start on a laptop with no network). The schema is applied idempotently at start-up, so adding `zones` and `zone_history` to `SCHEMA` is the migration: it is additive, non-destructive, and safe to run against an existing database. Confirmed by restarting the live backend against the pre-existing dev DB without data loss.

---

4. Part D — Dataset inventory and EDA

No new EDA numbers are invented here. Every figure below is read from a shipped file or was computed from file listings this round.

4.1 What exists

Dataset	Path	Reality	Used by
UCI HAR (activity)	`ml/data/har/` (`UCI HAR Dataset.zip` 60,999,314 B; extracted tree)	Real. `X_train` 7,352×561, `X_test` 2,947×561, 561 features, 128-sample raw window @ **50 Hz**, 9 raw channels	`ml/har_preprocess.py` → `ml/har_train.py` (activity) and `ml/har_fall.py` (impact detector)
UniMiB SHAR (future risk)	`ml/data/unimib/UniMiB-SHAR.zip` 266,206,080 B; `data/full_data.mat` 29,084,022 B	Real. Read via `full_data.mat`, **not** the pre-cut 151-sample windows	`ml/future_risk/dataset.py` → `ml/future_risk/train.py`
GeoLife (GPS)	`ml/data/geolife/…` 18,670 `.plt` files, 182 user dirs, ~1.67 GB; raw zip 313,164,406 B	Real. Labels are **transportation mode only** (`walk, bike, bus, car, …`) — no danger/incident labels	`ml/eda.py`, `ml/transport_mode.py`, `ml/history.py`
OSM (map)	`ml/data/osm/tiananmen_1p5km_overpass.json` 3,039,563 B, 15,980 elements	Real extract	`ml/osm_eda.py`, `ml/history.py`
Open-Meteo (weather)	`ml/data/weather/open-meteo_*.json`	Real extracts	`ml/weather_eda.py`
Synthetic fixture	`ml/data/synthetic_sample/SYNTHETIC_*`	**SYNTHETIC**, generated by `ml/data/make_synthetic_fixture.py` (seed 42), labelled with `SYNTHETIC_` names and a `SYNTHETIC_DATA_NOTE.txt`. Contains deliberate defects (teleport, null island, bad latitude)	tests only, as a fallback when the 299 MB download is absent

Absent, and stated as absent: there is **no tourist-incident / injury / danger-outcome dataset** anywhere in `ml/data`. Consequently **no supervised model was trained to predict tourist incidents**. The GeoLife labels are transportation mode only. The 0–100 approach score is a hand-set heuristic (§6), not a learned quantity.

4.2 EDA scripts that already exist (no notebook anywhere — `**/*.ipynb` returns none)

Script	Writes to
`ml/eda.py` (20,638 B)	`ml/outputs/eda/`
`ml/har_eda.py` (12,454 B)	`ml/outputs/har/eda/`
`ml/osm_eda.py` (20,668 B)	`ml/outputs/osm/`
`ml/weather_eda.py` (26,796 B)	`ml/outputs/weather/`

Script	Writes to
`ml/history.py` (21,150 B)	`ml/outputs/history/`
`ml/transport_mode.py` (19,985 B)	`ml/outputs/transport/`

Figures produced by these scripts live under `ml/outputs/**/figures`. **I did not re-run the EDA this round** (each script re-reads multi-hundred-megabyte archives); the plots and numbers already present in `ml/outputs` are the shipped ones.

4.3 Leakage controls (read from the code)

- **Future risk:** subject-disjoint split. 30 subjects → train 18, validation 6, test 6, seed 42. `metrics.json.split.leakage_check_subjects_disjoint = true`. Windows from one subject never straddle two splits.
- **Activity:** the dataset's own subject split — 21 train subjects / 9 test subjects, `subject_overlap: []`.
- **GPS anomaly:** grouped by `traj_uid` (`ml/models/split.json`, 900/300 trajectories).
- Windows inside one trial are **not** independent (a 50-sample window at a 10-sample stride overlaps its neighbours); the metrics report therefore *also* gives trial-level sensitivity and a false-alarm rate per minute rather than only window-level figures, which is the honest way to read it.

---

5. Part E — The models, the formulas and the viva explanation

5.1 Activity recognition (Card 2)

- Artifact: `app/src/main/assets/har_activity_model.json`, 268,734 B, `random_forest`, **32 trees**, 24 time-domain features, 6 classes `WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING, LAYING`.
- Window 128 samples @ 50 Hz = 2.56 s; 6 channels (total acc x/y/z, gyro x/y/z).
- Documented metrics: accuracy **0.8351**, macro-F1 **0.8333** (shipped 24-feature model). The 561-feature report model reaches 0.9287 but is not the shipped on-device model.
- The repo also ships `har_fall_model.json` (classes `NO_FALL` / `FALL`, 40 trees) whose positives were **injected into real HAR windows** — the caveat is written into `har_fall_report.json.caveat`, so it must not be quoted as a field-validated fall detector.

5.2 Future risk (Card 3) — what is predicted

- Artifact: `app/src/main/assets/future_risk_model.json`, 1,974,297 B (byte-identical copy at `ml/outputs/future_risk/future_risk_model.json`).
- `model_version`: `future-risk-unimib-v1`; `horizon_seconds` 1.0; `horizon_samples` 50; `window_samples` 50; `sample_rate_hz` 50; classes `["NO_IMMINENT_FALL","IMMINENT_FALL"]`; 22 features.
- **Target (this is the "valid future-fall target" question).** In `ml/future_risk/dataset.py`, a fall trial's impact is `t_impact = argmax(magnitude)`, accepted only if the peak is ≥ **2.0 g**. A **positive** window is one that **ends 1–50 samples (0.02–1.00 s) *before*** `t_impact`. A **negative** window is cut at the same cadence from an **ADL** (non-fall) trial. So the label is: "*does a real recorded hard-fall impact land within the next 1.0 s?*" This is a genuine forward-looking target, not a relabelling of the fall itself, and the pre-cut windows that are centred on the impact are deliberately **not** used (they would leak the impact into the input).

5.3 The exact probability formula

The on-device forest is walked by `DecisionForest` (a node goes **left when** `value <= threshold`, exactly as scikit-learn does). Raw forest probability of the positive class:

```
p_raw = (1 / T) * SUM over trees t of ( n_{t,IMMINENT_FALL} / n_t )
```

where `n_{t,IMMINENT_FALL}` are the training samples of the positive class that fell into the leaf reached in tree `t`, and `n_t` the total training samples in that leaf. That is `RandomForestClassifier.predict_proba`.

That vote share is **not** a probability — with `class_weight="balanced_subsample"` it is systematically over-confident. The number shown to the tourist is the **Platt-calibrated** value:

```
calibrated = 1 / (1 + exp( -(a * ln(p_raw / (1 - p_raw)) + b) ))
a = 1.24940294      b = -1.07904853
```

fitted on the **validation** subjects only. Bands, read from the artifact:

```
HIGH      if calibrated >= 0.314046
MODERATE  if calibrated >= 0.069201
LOW       otherwise
```

`0.314046` is the threshold that maximised F1 on validation; `0.069201` is the largest threshold whose validation recall was ≥ 0.95, capped at HIGH. Both are read from the JSON at runtime — nothing is hard-coded in the app (`FutureRiskModel.band`).

Input quality gates: fewer than `min_samples = 50` samples, or a magnitude above `max_plausible_g = 16.0`, yields **UNAVAILABLE** with a reason and **no band** — never a defaulted "Low" (`FutureRiskAnalyzer`).

5.4 Why the displayed probability varies (≈5.4 % / 5.6 % / 9.2 %)

Three independent reasons, all real:

1. **Each prediction is a different window of live data.** The card re-scores a fresh 1-second (50-sample) accelerometer window at the app's cadence. Two consecutive windows on a mostly-still phone differ slightly in noise, tilt and jerk, so `p_raw` differs slightly, so `calibrated` differs. 5.4 % and 5.6 % are two such neighbouring windows.
2. **The displayed number is post-calibration, not the vote share.** The logistic map with `a = 1.2494`, `b = -1.0790` spreads values in the low region, so small `p_raw` differences can show up as visible percentage differences.
3. **The band tracks that number arithmetically.** Two of the observed values (5.4 %, 5.6 %) are **below** `0.069201`, so the card reads *"No sign of an imminent hard fall"*. **9.2 % is above** `0.069201` and therefore, by the shipped rule, falls in **MODERATE** — the card would read "Moderate" for that window. I can state this arithmetic exactly; I cannot know which quantity any particular historical screenshot was rendering (calibrated probability vs. a raw score) without that screenshot, and I will not guess.

Reported performance (documented in `docs/future_risk_prediction.md` §9, backed by `ml/outputs/future_risk/metrics.json`). Held-out test = 10,415 windows / 6 unseen subjects / 1,339 positive:

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Majority class	0.0000	0.0000	0.0000	0.5000	0.128565
Logistic regression	0.4760	0.8432	0.6085	0.904389	0.659633
Hist gradient boosting	0.7916	0.6721	0.7270	0.945913	0.815000
RF raw @0.5	0.6534	0.7827	0.7122	0.936848	0.764569
Shipped: RF calibrated, HIGH ≥ 0.314046	**0.6707**	**0.7498**	**0.7080**	**0.936848**	**0.764569**

Confusion matrix TN 8,583 / FP 493 / FN 335 / TP 1,004; accuracy **0.9205**; Brier 0.056675 (calibrated) vs 0.062600 (raw). Trial-level sensitivity **0.929577** (264 of 284 held-out fall trials flagged). ADL false-alarm rate **5.09 per minute** at the app's 0.64 s cadence (16.30/min at the model's 0.2 s stride). `hist_gradient_boosting` **is genuinely better on PR-AUC (0.815000) but is not shippable on-device** as a hand-written JSON forest — that trade-off is stated in the repo and is the honest answer to "why not the best model?".

---

6. Part F — Four separate outputs, each labelled honestly

Output	Nature	Label on screen
Geographical safety	Deterministic geometry	"your position", "not a fall-risk score"
Activity / movement	Trained (UCI HAR, subject-disjoint)	"trained model"
Future risk	Trained (UniMiB SHAR, subject-disjoint, calibrated)	"trained model", with thresholds and model version
Approach / historical risk (`ApproachRiskModel`, 0–100)	**Hand-set weighted checklist**, explicitly *not* learned	`ApproachRisk.kt` comment: "a published weighted score, not a learned model"

`ApproachRiskModel` weights (out of 100): severity DANGEROUS 40 / RESTRICTED 28 / SAFE 5; recent crossings 3 each capped at 18 (or 4 for never-visited); proximity fraction × 16; night 12 / dusk 5; ambulating 4; live impact score 12 above 0.55 (6 when unavailable); total coerced to `[0, 100]`; bands < 30 LOW, < 60 MODERATE, else HIGH; approach radius 200 m. It is a separate Kotlin type from `FutureRiskBand`, so a screen cannot render the heuristic's band under the trained model's label.

---

7. Part H — Tests, acceptance and what is NOT done

7.1 Tests actually run this round

Suite	Result
`:app:testDebugUnitTest` (Android JVM)	**575 tests, 568 passed, 0 failures, 0 errors, 7 skipped, 53 classes** (was 548/541/50)
`python -m pytest backend\tests`	**285 passed** (was 265) — the 20 new ones are `backend/tests/test_zones.py`

Suite	Result
Live HTTP probe of the zone API	all 11 steps behaved as designed (§3.3)
APK contains the new code	verified by extracting all 17 dex entries and byte-searching for the new strings ("YOU ARE INSIDE A RESTRICTED AREA", "Geographical safety", "Restricted zones (authority)", "/api/v1/authority/zones") — all found

Not re-run this round: the `ml/` suites (last measured 179 passing) and the instrumented `androidTest` suites (no emulator or device is attached; `adb devices` is empty). I will not claim they passed.

7.2 New regression tests

Android (27 new tests in 3 new classes; the suite went 548 → 575 tests, 50 → 53 classes). - `GeoSafetyCardModelTest` (12): inside restricted → red verdict with the exact warning text; dangerous beats restricted in the headline; zone name + **distance to boundary** + accuracy + time are all present; boundary → explicit uncertainty naming the zone; coarse fix → uncertainty, not a guess; no fix while tracking → *checking*; no fix with tracking stopped → *unavailable*, never green; stale fix → *unavailable*; **a Low fall-risk prediction leaves the inside-zone warning intact**; the distinguishing sentence is present. - `TouristNavigationTest` (4): the tourist tabs are exactly Home/Tracking/Alerts/Settings; no tab is a zone-management route; `zones` and `editor` are authority-only and not tourist tabs; the editor route's default. - `AuthorityZoneApiTest` (11): zone parsing, lat/lon order preserved, malformed rows dropped, unknown `zoneType` degrades to the **safe-to-warn** direction, `{"zone":…}` single-zone answers, create-body round trip, and that the tourist read path differs from every write path.

Backend (`backend/tests/test_zones.py`, 20 tests). Officer create/list/edit/status/archive; revision bumping and history recording; **tourist 403 on create / edit / status / delete with the row unchanged**; 401 with no token; tourist may *read* active zones; fewer than 3 vertices rejected; out-of-range latitude rejected; degenerate polygon rejected; non-numeric coordinate rejected; unknown zone type rejected; blank name rejected; an officer edit is what the tourist next reads; a draft is not published; zone changes leave the event audit trail untouched.

7.3 Acceptance checklist (my restatement of the 12 criteria, with status)

#	Criterion	Status
1	Three separate concepts clearly presented	✅ Card 1 added; Cards 2 & 3 already existed and are now visually separated
2	Geographical status prominent and independent of ML	✅ Card 1 under the banner on Home and Tracking; no model input
3	Uncertainty state when GPS accuracy is insufficient	✅ `NEAR_BOUNDARY_UNCERTAIN` / `LOCATION_UNAVAILABLE`
4	A low ML score never suppresses the geographical warning	✅ Structurally (no data path) **and** pinned by test
5	Point-in-polygon, lat/lon order, boundary policy verified	✅ Inspected + existing suite passing; no defect found in the geometry
6	No green "safe" while a restricted-zone warning is active	✅ Banner and Card 1 share one state; green requires a current fix outside
7	Tourist Create Zone and all zone controls removed	✅ Nav + Home changed; guarded by test

#	Criterion	Status
8	Authority-only create/edit/delete/activate/archive	✅ Backend API + live 403/401 proof; app screens officer-gated
9	Invalid/degenerate polygons rejected, with history	✅ 400 on degenerate; `zone_history` records CREATE/STATUS/ARCHIVE
10	Safe migration, no database reset	✅ Additive `CREATE TABLE IF NOT EXISTS`; existing data untouched on restart
11	Regression tests for the above	✅ 16 Android + 20 backend, all passing
12	APK built after tests pass, path reported	✅ §7.4

7.4 The APK

Built **after** the JVM and backend suites passed:

```
D:\TouristSafetyMonitoring-src\app\build\outputs\apk\debug\app-debug.apk
21,317,914 bytes (20.33 MB)
SHA-256  B42B945EE27A7580F205E73E0CB74F06C9B567BF63EFB7AF390410C006516684
```

Copies written to `D:\TouristSafetyMonitoring-src\app-debug.apk`, `C:\Users\DELL\Desktop\app-debug.apk` and `C:\Users\DELL\Downloads\app-debug.apk`. Build command:

```
powershell -NoProfile -ExecutionPolicy Bypass -File ".\build.ps1" ":app:assembleDebug" \
  "-PAUTHORITY_URL=http://192.168.1.102:8080" "--console=plain" "--no-daemon"
```

7.5 NOT done — stated plainly

1. **The app does not yet pull authority zones into its local store, and does not push officer edits to the backend.** The backend is authoritative and fully tested; the app can read it (`AuthorityClient.activeZones/authorityZones/createZone/setZoneStatus`, all unit-tested); but the local `geo_fences` table has no server id, so a correct sync needs a new column plus `MIGRATION_3_4`. I did **not** ship that migration because it can only be verified by an instrumented `MigrationTestHelper` run, and there is no emulator or device attached. Shipping an unverified Room migration that could crash the app on upgrade would violate "no destructive/faked changes". **Consequence today:** the app's own officer screens still write to the local copy; the backend API is what a non-app client (the web dashboard, `curl`) must use, and it is where the 403 is enforced. This is the single largest remaining gap.
2. **No on-device verification.** No emulator, no physical device (`adb devices` empty). The APK is built and its contents verified by dex inspection, but the new cards have not been *seen* rendered on a screen. The offline banner clearing, the HTTP round trip from the phone, and the officer console's zone button are therefore **NOT EXECUTED**.
3. **The map-editor screens still operate on the local zone copy.** They are authority-gated, but until (1) is done their writes do not reach the server.
4. **`ml/` suites were not re-run** this round; the ML work here was audit-only (inventory + reading the shipped artifacts), and no ML code was changed.
5. **No new EDA plots** were generated (the scripts exist; re-run them if fresh figures are needed for the viva).
6. All changes are **uncommitted**; there is no git repository here.

---

8. Changed files

Backend - `backend/tsm/db.py` — added `zones` and `zone_history` (additive, idempotent) to `SCHEMA`. - `backend/tsm/api.py` — `import math`; new authority-owned zones section: `_parse_vertices`, `_polygon_area_square_meters`, `zone_json`, `_record_zone_change`, and the 7 routes. - `backend/tests/test_zones.py` — **new**, 20 tests.

Android — main - `ui/components/GeoSafetyCardModel.kt` — **new**: the 5 states, the exact warning text, pure model. - `ui/components/GeoSafetyCard.kt` — **new**: Card 1 composable. - `ui/components/SafetyVisuals.kt` — added `geoSafetyVisualsFor(state)`. - `ui/screens/HomeScreen.kt` — Card 1 added; banner now geographical; zone card read-only; `onCreateZone` / `onOpenZones` removed. - `ui/screens/TrackingScreen.kt` — Card 1 added; banner now geographical; sections split ("Movement and fall-risk models"). - `ui/screens/OfficerScreen.kt` — `onManageZones` parameter + "Restricted zones" button. - `navigation/AppNavigation.kt` — bottom bar is Home/Tracking/Alerts/Settings; `TouristNavigation` object; `AuthorityRequired` gate; zone routes officer-gated; titles updated. - `authority/AuthorityContract.kt` — `ZONES`, `AUTHORITY_ZONES`, status/edit path builders. - `authority/AuthorityZone.kt` — **new**: the authority zone type and wire constants. - `authority/AuthorityApi.kt` — `parseZones`, `parseZone`, `zoneBody`, `zoneStatusBody`. - `authority/AuthorityClient.kt` — `activeZones`, `authorityZones`, `createZone`, `setZoneStatus`.

Android — tests - `test/.../ui/components/GeoSafetyCardModelTest.kt` — **new**. - `test/.../navigation/TouristNavigationTest.kt` — **new**. - `test/.../authority/AuthorityZoneApiTest.kt` — **new**.

Migrations - Local Room: **none.** `AppDatabase` stays at version 3. (Adding the sync column is the documented next step, §7.5.) - Backend: nothing to run — the new tables are created idempotently at start-up.

---

9. How to reproduce

```
# Backend tests
cd D:\TouristSafetyMonitoring-src; python -m pytest backend\tests -q

# Backend (live)
python backend\run.py --host 0.0.0.0 --port 8080

# Android JVM tests, then the APK
powershell -NoProfile -ExecutionPolicy Bypass -File ".\build.ps1" ":app:testDebugUnitTest" "--console=plain
powershell -NoProfile -ExecutionPolicy Bypass -File ".\build.ps1" ":app:assembleDebug" "-PAUTHORITY_URL=htt

# Live zone-API probe (writes then archives one dev zone)
python C:\Users\DELL\AppData\Local\Temp\opencode\probe_zones.py
```